

1a Prova de ELT024 – Programação para Sistemas Embarcados 2014

Prof. Rodrigo Maximiano Antunes de Almeida

(10 pts) Questão 1: Qual a função de um loop infinito no main?**Resposta:** Garantir que o código dentro do loop repita-se até o equipamento ser desligado;

Não deixar com que o a função main termine e o equipamento resete ou tenha algum problema.

(15 pts) Questão 2: Qual a diferença entre as operações lógicas e bitwise na linguagem C? Apresente o resultado das seguintes operações:**result1 = 0x77 & 0x88;**
result2 = 0x77 && 0x88;**result3 = 0xAA | 0x55;**
result4 = 0xAA || 0x55;**result5 = !0x0F;**
result6 = ~0x0F;**Resposta:** Nas operações Lógicas os operandos são tomados como um todo, nas operações Bitwise os operandos são processados bit à bit.**result1 = 0x00; //ou 0**
result2 = 0x01; //ou 1**result3 = 0xFF; //ou 255**
result4 = 0x01; //ou 1**Result5 = 0x00; //ou 0**
result6 = 0xF0; //ou 240**(20 pts) Questão 3:** Configure os registros INTCON e RCON para que apenas a interrupção do periférico TIMER0 funcione. O sistema de interrupção deve funcionar em modo de compatibilidade. Obs.: todos os bits dos registros podem ser habilitados colocando-se o valor 1 (um) e desabilitados colocando-se o valor (0). Apresente os valores em binário, decimal e hexadecimal.

INTCON = 0b_____;

```
//      |||||  L-> RBIF: Flag de interrupção da porta B
//      |||||  L-> INT0IF: Flag de interrupção externa 0
//      |||||  L-> TMR0IF: Flag de interrupção do timer 0
//      |||||  L-> RBIE: Bit que habilita interrupção na porta B
//      |||||  L-> INT0IE: Bit que habilita interrupção externa
//      |||||  L-> TMR0IE: Bit que habilita interrupção do Timer 0
//      |  L-> PEIE/GIEL: p/ IPEN = 1: habilita interrupções de baixa prioridade
//      |  L-> p/ IPEN = 0: habilita interrupções dos periféricos
//      |  L-> GIE/GIEH: p/ IPEN = 1: habilita interrupções de alta prioridade
//      |  L-> p/ IPEN = 0: habilita todas as interrupções
```

RCON = 0b_____;

```
//      |||||  L-> BOR: Flag de brown-out reset
//      |||||  L-> POR: Flag de power-on reset
//      |||||  L-> PD: Flag de detecção de power-down
//      |||||  L-> TO: Flag de detecção do watchdog
//      |||||  L-> RI: Flag de reset por software
//      |||||  L-> não usado
//      |  L-> BOR: habilitação de detecção do Brown-out
//      |  L-> IPEN: sistema de interrupção em modo de prioridade(1) ou compatibilidade(0)
```

Resposta: INTCON = 0x1110 0xxx, E0, 224. RCON = 0xxx xxxx, 0, 0**(25 pts) Questão 4:** Um modelo simples de cifragem é a utilização de uma operação XOR dos dados a serem cifrados com um número. Para realizar a decifragem dos dados basta fazer novamente a operação XOR com o mesmo número. Este número pode ser entendido como uma senha. Este sistema apresenta um nível de segurança muito ruim. No entanto é bastante rápido, sendo útil em algumas situações. Supondo que a região onde os dados estão gravados na memória vai de 0x100 até 0x2FE, faça um **programa** que cifre estes dados realizando a operação de XOR com o número **0xEF** e armazene os dados cifrados na região de memória de 0x300 até 0x4FE.

//exemplo de cifragem

dadoCifrado = dadoNãoCifrado ^ 0xEF;

Resposta:

```
void main (void){
    char * pt_origem;
    char * pt_destino;
    int i;
    for (i = 0x100; i< 0x2ff. I++){
        pt_origem = i;
        pt_destino = i + 0x200;
        *pt_destino = (*pt_origem) ^ 0xEF;
    }
    for(;;);
}
```

(30 pts) Questão 5: Um problema comum é a utilização de um recurso ou periférico por duas partes do programa como por exemplo um equipamento com dois sensores que querem escrever seus dados no mesmo LCD. Se essa escrita não for sincronizada, os algarismos dos valores dos sensores poderão aparecer embaralhados. Uma solução é utilizar uma estrutura conhecida como **mutex**. O mutex pode ser implementado por uma variável inicializada com o valor zero. Quando um dos sensores quiser escrever no LCD ele verifica se o mutex está zero, se estiver em zero ele muda o valor do mutex pra um (1), ou seja, reserva o mutex. Isso sinaliza que o LCD está em uso. Quando terminar de utilizar o LCD ele retorna o valor do mutex para zero(0). Se o outro sensor quiser escrever no LCD enquanto o mutex estiver em um (1) ele deverá esperar o mutex voltar para zero (0) antes de começar. Faça uma **biblioteca** "mutex" com 3 funções: InicializaMutex() → responsável por inicializar as variáveis da biblioteca; ReservaMutex() → verifica se o mutex está disponível e faz a reserva dele retornando um valor que indica se a reserva foi feita com sucesso (1) ou não (0); e DevolveMutex() → retorna o mutex ao seu estado normal. A biblioteca deve salvar o estado do mutex numa variável interna.

```
//mutex.h
#ifndef MUTEX_H
#define MUTEX_H
    void InicializaMutex(void);
    char ReservaMutex(void);
    void DevolveMutex(void);
#endif

//mutex.c
static char mutex;
void InicializaMutex(void){
    //inicializa livre;
    mutex = 0;
}
char ReservaMutex(void){
    if (mutex == 0){
        mutex = 1;
        return 1;//conseguiu reservar
    }
    return 0; //não conseguiu
}
void DevolveMutex(void){
    if(mutex == 1){
        mutex == 0;
    }
}
```